

Appendix A: C Code

```
1  #include <math.h>
2  #include <stdio.h>
3  #include "mkl.h"
4
5  /* change name of matmul# function to matmul depending on which one to execute */
6
7  /* naive version of matrix multiplication C=A*B */
8  void matmul(int maxsize, int size, double **a, double **b, double **c)
9  {
10     int i,j,k;
11     double scale;
12     for (i=0; i<size; i++) /* number of rows */
13     {
14         for (j=0; j<size; j++) /* along each row */
15         {
16             c[i][j] = 0.0;
17             for (k=0; k<size; k++) /* across row for a, and down column for b */
18             {
19                 c[i][j] += a[i][k]*b[k][j];
20             }
21         }
22     }
23 }
24
25
26 /* loop restructure (Dowd Severance p159-160 */
27 void matmul2(int maxsize, int size, double **a, double **b, double **c)
28 {
29     int i,j,k;
30     double scale;
31
32     for (i=0; i<size; i++) /* number of rows */
33     {
34         for (j=0; j<size; j++) /* along each row */
35         {
36             c[i][j] = 0.0;
37         }
38     }
39
40     for (k=0; k<size; k++) /* across row for a, and down column for b */
41     {
42         for (i=0; i<size; i++) /* number of rows */
43         {
44             scale = b[i][k];
45
46             for (j=0; j<size; j++) /* along each row */
47             {
48                 c[i][j] += a[k][j]*scale;
49             }
50         }
51     }
```

```

52 }
53
54 /* blocking (Dowd Severance p161-165 */
55 void matmul3(int maxsize, int size, double **a, double **b, double **c)
56 {
57     int i,ii,j,jj,k,kk,blocksize;
58
59     /* initialize c */
60     for (i=0; i<size; i++)
61     {
62         for (j=0; j<size; j++)
63         {
64             c[i][j] = 0.0;
65         }
66     }
67
68     /* tried to do Dowd Severance methods, but couldn't get it to work */
69     /*for (i=0; i<size; i++) */
70     /*{*/
71         /*for (j=0; j<size; j++) */
72         /*{*/
73             /*c[i][j] = 0.0;*/
74
75             /*for (k=0; k<size/2; k++) */
76             /*{*/
77                 /*c[i][j] += a[i][k]*b[k][j];*/
78             /*}*/
79         /*}*/
80     /*}*/
81     /*for (i=0; i<size; i++) */
82     /*{*/
83         /*for (j=0; j<size; j++) */
84         /*{*/
85             /*c[i][j] = 0.0;*/
86
87             /*for (k=size/2; k<size; k++) */
88             /*{*/
89                 /*c[i][j] += a[i][k]*b[k][j];*/
90             /*}*/
91         /*}*/
92     /*}*/
93
94     /* method from reading algorithms on the internet */
95     /* tried multiple ways to get this blocking to work */
96     /* but none resulted in a very fast algorithm */
97     blocksize = 5;
98     for (i=0; i<size; i+=blocksize)
99     {
100         for (j=0; j<size; j+=blocksize)
101         {
102             for (k=0; k<size; k+=blocksize)
103             {
104                 for (ii = i; ii < i+blocksize; ii++)
105                 {

```

```

106         for (jj = j; jj < j+blocksize; jj++)
107         {
108             for (kk = k; kk < k+blocksize; kk++)
109             {
110                 c[ii][jj] += a[ii][kk] * b[kk][jj];
111             }
112         }
113     }
114 }
115 }
116 }
117 }
118
119 /* intel mkl library dgemm */
120 void matmul4(int maxsize, int size, double **a, double **b, double **c)
121 {
122     int m, n, k;
123     double alpha, beta;
124
125     /* alpha * a[mxk] * b[kxn] = beta * c[mxn] */
126     m = size;
127     n = size;
128     k = size;
129     alpha = 1.0; /* scale a*b */
130     beta = 1.0; /* scale c */
131
132     cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
133                 m, n, k, alpha, *a, k, *b, n, beta, *c, n);
134 }

```

Appendix B: Matlab Code

```
1 % Andrew Navratil
2 % CE 791 HPC – HW 3
3
4 close all; clear all; clc;
5
6 % use readmatrix since file has column names at top
7 % modified driver.c to include tab delimiter
8 m = 'henry'; % machine name: bezier, neer, henry
9
10 d = readmatrix(['hw3data/matmulc_' m '_ofast.dat']);
11 plot(d(:,1),d(:,2),'+k','MarkerSize',10,'DisplayName','naive loop w 0fast');
12 hold on;
13
14 %d = readmatrix(['hw3data/matmulc_' m '_o1.dat']);
15 %plot(d(:,1),d(:,2),'+b-','MarkerSize',10,'DisplayName','naive loop w 01');
16
17 %d = readmatrix(['hw3data/matmulc_' m '_o2.dat']);
18 %plot(d(:,1),d(:,2),'+b-','MarkerSize',10,'DisplayName','naive loop w 02');
19
20 %d = readmatrix(['hw3data/matmulc_' m '_o3.dat']);
21 %plot(d(:,1),d(:,2),'+b--','MarkerSize',10,'DisplayName','naive loop w 03');
22
23 d = readmatrix(['hw3data/matmulc_' m '_mkl.dat']);
24 plot(d(:,1),d(:,2),'xk-','MarkerSize',10,'DisplayName','intel mkl dgemm');
25
26 d = readmatrix(['hw3data/matmulc_' m '_restruct.dat']);
27 plot(d(:,1),d(:,2),'*k--','MarkerSize',10,'DisplayName','loop restructure');
28
29 d = readmatrix(['hw3data/matmulc_' m '_blocking.dat']);
30 plot(d(:,1),d(:,2),'ok-','MarkerSize',10,'DisplayName','blocking');
31
32 xlabel('Matrix Size');
33 ylabel('Mflops');
34 title(m);
35 %title([m ' (2)']);
36 set(gca,'FontSize',18);
37 legend;
```